

面向对象程序设计

面向对象

对象（Object）

- 对象是由数据及能对其实施的操作（成员函数）所构成的封装体，属于值的范畴。
- 直接方式：定义一个类型为类的变量，即对象。对象的作用域、生存期与普通变量相同。通过对象名来标识和访问。
- 例如：

```
class A
```

```
{ ... };
```

```
A a1;      //创建一个A类的对象
```

```
A a2[100]; //创建由100个A类对象组成的数组
```

- 间接方式（动态对象）：在程序运行时，通过**new**操作或**malloc**库函数来创建对象。

其内存空间在程序的堆区中

动态对象通过指针来标识和访问

动态对象用**delete**操作或**free**库函数来撤消

例如：

```
```cpp
```

```
 A *p;
```

```
 p = new A[100]; //或 p = (A*)malloc(sizeof(A)*100);
```

```

```

```
 delete []p; // 或free(p);
```

对于动态对象，需注意下面两点：

- 用new创建的动态对象数组只能用默认构造函数进行初始化。delete中的“[]”不能省，否则，只有第一个对象的析构函数会被调用。
- 函数malloc不会调用对象类的构造函数。函数free也不会调对象类的析构函数。

## 类 (Class)

- 类（对象的类型）描述了一组具有相同特征的对象，属于类型的范畴。
- C++的类是一种用户自定义类型。对象构成了面向对象程序的基本单位，而对象的特征则由相应的类来描述。
- 定义的形式如下：

```
class <类名> { <成员描述> };
```

其中，类的成员包括：数据成员、成员函数

- 类的定义构成了一个作用域——类作用域。
- 在其中定义的标识符只能在类的内部访问
- 当类使用与其成员重名的全局标识符时，需要在该全局标识符前面加上 ::
- 数据成员的定义格式与非成员数据的定义格式相同，但定义数据成员时不允许进行初始化。（在构造函数中初始化）

例如：

```
```cpp
class A

{   int x=0;   //Error

    const double y=0.0;   //Error

    .....

};
```
```

- 数据成员的类型可以是任意的C++类型（void除外）
- 但在定义一个数据成员的类型时，如果该类型的定义未见到或未定义完，则该数据成员的类型只能是指针或引用类型。

例如：

```
```cpp
class A;   //A是在程序其它地方定义的类
```

```
class B
```

```

{  A a; //Error, 未见A的定义

    B b; //Error, B还未定义完

    A *p; //OK

    B *q; //OK

    A &aa; //OK

    B &bb; //OK  };
...

```

- 成员函数的定义格式和非成员函数的定义格式相同，并且可放在类定义的内部或外部。位于类定义的外部时，函数名需要使用 <类名> :: 来受限。

例如：

```

class A
{
    ...

    void f() { ... } //1. 位于类定义的内部
};

class A
{
    ...

    void f();

};

void A::f() { ... } //2. 位于类定义的外部

```

- 类的成员函数是可以重载的（析构函数除外），它遵循一般函数的重载规则。

例如：

```
```cpp
```

```
class A
{

 public:

 void f();

 int f(int i);

 double f(double d);

};
...
```

- C++使用成员访问控制修饰符来限制对成员的访问：
- **public**：成员的访问不受限制，即构成了类提供给外界使用的接口。
- **private**：成员只能在本类或友元中访问。类的内部使用的数据成员、成员函数应该指定为 **private**。
- **protected**：成员只能在本类、友元或派生类中访问。